

Project Executable Files

Date	29 September 2026
Team ID	SWTID-2026-5955
Project Name	PocketSmart AI - Your Smart Budget & Recommendation Assistant
Maximum Marks	3 Marks

This section requires submission of all executable/project files related to the developed solution. Ensure that all files are properly organized within the submission folder and are functional. The evaluator must be able to run and test the project using the provided files and instructions without additional setup beyond what is documented in the README/setup guide.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes/No/NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide	Yes
3	requirements.txt / dependency file	Yes
4	Database schema / seed files	Yes - SQULE + PostgreSQL schema
5	Environment config file (.env.example)	Yes - GOOGLE_API_KEY example
6	Deployed application URL	Yes - IBM Cloud URL
7	APK / Executable binary	NA - Web app
8	Dockerfile / Containerization config	Yes - Docreofile included
9	Test files and test results	Yes - Locust, Postman results
10	Demo video or walkthrough	Yes - Loom link

PocketSmart AI — Project Executable Files

Page 2

Step 2 | File / Folder Structure

/PocketSmart-AI-root

- /src — Source code files
 - | ---< app.py, main.py, gemini_uutls.py
- /routes — /generate-home, /generate-party, /generate-jewelry, /login, /register, /history
- /services — Gemini AI logic, budget allocation
- /models — User models, auth models
- /templates — Jinja2 HTML — home_planner.html, party_planner.html, jewelry_planner.html, dashboard.html, login.html
- /static — CSS, JS, images
- /tests — Locust load tests, Postman collection
- /docs — Documentation PDFs
- /config — Configuration files
- README.md — Setup and run instructions
- requirements.txt — Python dependencies (Flask, FastAPI, google-generativeai, Pillow, bccrpt, python-dotenv)
- .env.example — GOOGLE_API_KEY, JWT_SECRET, DB_URL
- {-- Dockorfile — IBM Cloud deployment.

Step 3 | Deployment Access Details

Field	Details
Hosted/Deployed URL	https://pocketsmart-ai-2026-5955.onrender.com (IBM Cloud)
Login Credentials (Demo)	demo@pocketsmart.ai / Demo@12345
Platform/Hosting Provider	IBM Cloud / Render / Docker
Repository Link	https://github.com/SWTID-2026-5955/PocketSmart-AI
Demo Video Link	https://loom.com/share/pocketsmart-ai-demo-29sep2026

Step 4: Run Instructions | Step 5: Known Issues

✓ Step 4: Run Instructions

Pre-requisites:

- Python 3.11+ • Flask, FastAPI • Google Cloud account with Gemini API key
- 4GB RAM minimum

Execution Steps:

1. **Clone repository:** `git clone https://github.com/SWTID-2026-5955/PocketSmart-AI`
2. **Install dependencies:** `pip install -r requirements.txt` (Flask, FastAPI, uvicorn, google-generativeai, Pillow, bcrypt, python-dotenv, Flask-Session)
3. **Configure environment:** `copy .env.example` to `.env` and add `GOOGLE_API_KEY=your_key`, `JWT_SECRET`, `DB_URL`
4. **Run database:** SQLite auto-creates; for PostgreSQL run migrations
5. **Start application:** `python app.py` or `uvicorn main:app --reload --port 5000`
6. Open browser at `http://localhost:5000`, login with `demo@pocketsmart.ai / Demo@12345`, test Home • Party • Jewelry • planners.

⚠ Step 5: Known Issues

S.No	Known Issue / Limitation	Workaround / Status
1	Gemini API rate limit under spike (>100 users)	Implemented caching, fallback mock recommendations
2	Jewelry image upload large files >5MB slow	Added resize to 512x512 and 5MB limit validation
3	No real-time e-commerce scraping	Currently mock + Gemini generation; future integration with Amazon/Flipkart affiliate APIs planned